

SELLER AND INVENTORY MANAGEMENT SYSTEM

Introduction:

An e-commerce platform manages products from different sellers and keeps track of the available stock. The Seller and Inventory Management System is designed to store seller information and monitor the quantity and availability of products.

The system helps to maintain seller details, connect sellers with products, and keep inventory information updated. It also provides useful reports about sellers, products, and stock levels.

Seller Table Creation:

The Seller table stores the basic information of sellers who provide products through the e-commerce platform. It contains seller ID, seller name, email, phone number, and address

Inventory Table Creation:

The Inventory table is used to maintain the stock details of products. It stores the inventory ID, product ID, seller ID, stock quantity, stock status, and last updated date.

```
SQL> CREATE TABLE Seller(seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone NUMBER(10), Address VARCHAR(45));  
Table created.  
  
SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7), Product_ID VARCHAR(7), Seller_ID VARCHAR(7), Stock_Quantity number, Stock_status VARCHAR(11), Last_Updated DATE);  
Table created.
```

Relationship Between Tables:

The Seller and Inventory tables are connected with the Product table to maintain proper data relationships.

Seller – Product Relationship

A seller can provide multiple products. Therefore, the relationship between Seller and Product can be represented as one-to-many.

Product – Inventory Relationship

Inventory stores the stock information related to each product. The Product and Inventory relationship is represented as one-to-one in this system.

Defining Primary Keys:

Primary keys are added to uniquely identify each record in the Seller and Inventory tables.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);  
Table altered.
```

Defining Foreign Keys:

Foreign keys are used to establish connections between the Inventory table and the existing Product and Seller tables.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

Unique Email Constraint:

Each seller should have a unique email address.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

NOT NULL Constraint:

The seller email field is made mandatory.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

Stock Quantity Validation:

The stock quantity should not contain a negative value.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK (Stock_Quantity >=0);  
Table altered.
```

Adding Seller Records:

Seller details are inserted into the Seller table using the following query:

```
SQL> INSERT INTO Seller VALUES('S00001', 'Vani Enterprises', 'mdvanienter@gmail.com', 8765432109, '1, Anna Nagar, Chennai');
1 row created.

SQL> INSERT INTO Seller VALUES('S00002', 'SD & CO', 'sellerD827@gmail.com', 9876543012, '62, Thirumal Nagar, Thanjavur');
Enter value for co:
old 1: INSERT INTO Seller VALUES('S00002', 'SD & CO', 'sellerD827@gmail.com', 9876543012, '62, Thirumal Nagar, Thanjavur')
new 1: INSERT INTO Seller VALUES('S00002', 'SD ', 'sellerD827@gmail.com', 9876543012, '62, Thirumal Nagar, Thanjavur')
1 row created.

SQL> INSERT INTO Seller VALUES('S00003', 'AtoZ', 'atozmarket@gmail.com', 7890654312, '90, George Town, Chennai');
1 row created.

SQL> INSERT INTO Seller VALUES('S00004', 'DSVV Manufacturers', 'dsvvmanuf142@gmail.com', 7824003009, '23, Thillai Nagar, Trichy');
1 row created.
```

```
SQL> select * from seller;
```

SELLER_	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S00001	Vani Enterprises 1, Anna Nagar, Chennai	mdvanienter@gmail.com	8765432109
S00002	SD 62, Thirumal Nagar, Thanjavur	sellerD827@gmail.com	9876543012
S00003	AtoZ 90, George Town, Chennai	atozmarket@gmail.com	7890654312
SELLER_	SELLER_NAME	EMAIL	PHONE
ADDRESS			
S00004	DSVV Manufacturers 23, Thillai Nagar, Trichy	dsvvmanuf142@gmail.com	7824003009

Adding Inventory Records:

Inventory records are created using the products already available in the Product table.

```
SQL> INSERT INTO Inventory VALUES('Inv0001', '501', 'S00001', 25, 'Available', TO_DATE('2026-09-12', 'YYYY-MM-DD'));
1 row created.

SQL> INSERT INTO Inventory VALUES('Inv0002', '502', 'S00002', 30, 'Available', TO_DATE('2026-09-13', 'YYYY-MM-DD'));
1 row created.

SQL> INSERT INTO Inventory VALUES('Inv0003', '503', 'S00003', 20, 'Available', TO_DATE('2026-09-14', 'YYYY-MM-DD'));
1 row created.

SQL> INSERT INTO Inventory VALUES('Inv0004', '504', 'S00004', 40, 'Available', TO_DATE('2026-09-15', 'YYYY-MM-DD'));
1 row created.
```

```
SQL> select * from inventory;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATU	LAST_UPDA
Inv0001	501	S00001	25	Available	12-SEP-26
Inv0002	502	S00002	30	Available	13-SEP-26
Inv0003	503	S00003	20	Available	14-SEP-26
Inv0004	504	S00004	40	Available	15-SEP-26

Seller wise product:

This report provides information about the number of products handled by each seller and the total stock available with them.

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
SD	1	30
DSVV Manufacturers	1	40
AtoZ	1	20
Vani Enterprises	1	25

Stock Availability Report:

This report displays the product name along with its current stock quantity and availability status.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;

no rows selected
```

Inventory Analysis:

Inventory analysis is performed to understand the overall stock position of the products.

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
4

Products Currently Out of Stock:

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';
```

OUTOFSTOCK_PRODUCTS
0

Product with Highest Stock:

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);
```

no rows selected

Average Stock Quantity:

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;
```

AVERAGE_INVENTORY
28.75

Conclusion:

The Seller and Inventory Management System provides an organized method for maintaining seller and stock information. The database uses primary keys, foreign keys, and constraints to maintain data consistency. The inventory reports help identify available products, stock quantities, out-of-stock products, highest-stocked products, and average inventory levels.